OWL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

OWL

https://github.com/mikel-egana-aranguren/DBA

CC BY

OWL

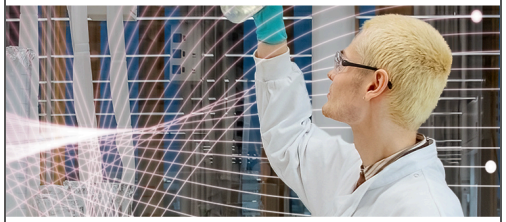
the LEGO Group

Lead Ontology Engineer - Semantic Data Modelling

- Proven expertise in Semantic Modelling and Ontology Design using open standards.
- Proficiency in RDF, OWL, and SHACL for developing and managing ontologies.
- Experience in data governance with knowledge graphs and semantic models.
- Strong programming skills in languages such as Python and TypeScript.
- Excellent communication and collaboration skills to work effectively within a multidisciplinary team.
- Understanding Domain Driven Design, loosely coupled components, and event driven architecture.
- Travel 15-20 days a year (dependent on need).
- Communicate effortlessly in English both written and verbally.

OWL

AstraZeneca Unirse a la Red de Talento



← Todos los Trabajos

Ontologist

Macclesfield, , United Kingdom

Aplicar Ahora | Añadir al Carro de Trabajo

Descripción del puesto vacante

Introduction to role

Join our Global Operations Data Office as an Ontologist and be part of a team that is transforming the pharmaceutical industry through data innovation.

Our mission is to ensure AI-ready data that supports our Operations 2030 ambition, enabling life-changing medicines. We partner with various business areas, including Pharmaceutical Technology and Development, Manufacturing, and Global Supply Chain, operating from hub sites in the UK, Sweden, US, China, and Global Technology Centres in India and Mexico. Here, your work will directly impact patients by empowering the business to perform at its peak through cutting-edge science and digital technology platforms.

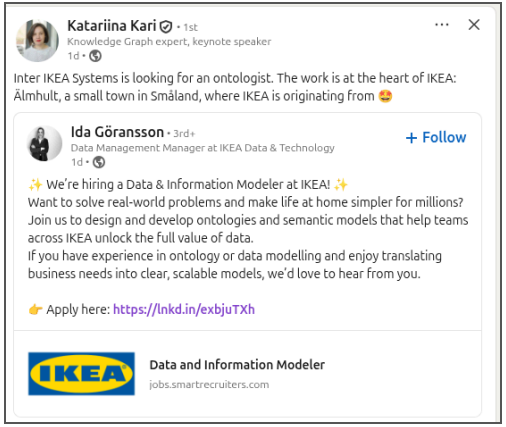
Accountabilities

As an Ontologist, you will play a crucial role in developing semantic data solutions across manufacturing, supply chain, and development functions. You will engineer robust ontology driven knowledge models, integrate structured and unstructured data, and apply semantic standards to enhance data interoperability. Collaborating closely with product teams and data engineers, you will bring semantic capabilities into real-world business applications. This is your chance to work at the forefront of semantic data technologies in the pharmaceutical industry—enabling FAIR data, accelerating AI use cases, and driving smarter digital operations.

Essential Skills/Experience

- Proficiency with RDF, SKOS, OWL, SHACL, SPARQL, and other semantic web technologies.

OWL


Katariina Kari • 1st
Knowledge Graph expert, keynote speaker
1d •
Inter IKEA Systems is looking for an ontologist. The work is at the heart of IKEA: Älmhult, a small town in Småland, where IKEA is originating from 🤩
Ida Göransson • 3rd+
Data Management Manager at IKEA Data & Technology
1d •
+ Follow
✨ We're hiring a Data & Information Modeler at IKEA! ✨
Want to solve real-world problems and make life at home simpler for millions?
Join us to design and develop ontologies and semantic models that help teams across IKEA unlock the full value of data.
If you have experience in ontology or data modelling and enjoy translating business needs into clear, scalable models, we'd love to hear from you.
👉 Apply here: https://lnkd.in/exbjuTXh
IKEA
Data and Information Modeler
jobs.smartrecruiters.com

OWL

OWL: Web Ontology Language

W3C-ren estandar ofiziala web-ean ontologiak sortzeko, semantika zehatz eta formal batekin (=HTML, !=SQL)

Ontologia: interes-eremu bati buruzko deskribapen zehatzen multzoa

Giza komunikazioan gaizki-ulertuak ekiditen ditu

Softwareak modu uniforme eta aurreikusgarrian jokatzea bermatu

OWL

RDF hizkuntza bera datuak eta bere hiztegia definitzeko* (NoSQL!RDF!)

1. funtzioa: datuen integraziorako hiztegia

2. funtzioa: interes-eremua deskribatzeko eta arrazonamendu automatikoa aplikatzeko lengoaia

OWL

Ez da programazio-lengoaia: deklaratiboa da, domeinuaren egoera bat deskribatzen du

Ez da eskema-lengoaia: ez du dokumentu baten egitura agintzen (adib. XML/JSON schema) ezta datuen egitura - Horretarako SHACL dago

Ez da datu-baseen framework bat: **Open World Assumption** jarraitzen du (ez dagoen informazioa egia izan daiteke, ez da faltsu bezala suposatzen)

Closed World vs Open World

Closed World Assumption (CWA): DBan ez dagoena faltsua da (SQL)

Open World Assumption (OWA): esaten ez dena egia edo faltsua izan daiteke (OWL)

Adibidea: Mariaren adinari buruzko informaziorik ez badago, SQLk NULL/adinik gabe suposatzen du; OWLek ez dakiela suposatzen du (Esplizituki esan behar da Mariak ez duela adinik)

No Unique Name Assumption

Bi URI ezberdinek entitate berari erreferentzia egin diezaiokete

ex:Bilbao eta ex:Bilbo owl:sameAs izan daitezke

Iturri ezberdinetako datuen integraziorako baliagarria

Arrazonamenduaren bidez bateratzea automatikoa ahalbidetzen du

owl:differentFrom bereizketa behartzeko

OWL

Entitateak: domeinuko objektuei erreferentzia egiten dieten elementuak: *Jon, Pertsona, adinaDu, ...*

Axiomak: ontologia baten oinarrizko baieztapenak (egiazkotzat hartzen dira): *Jon Pertsona bat da, Helduek 18 baino adin handiagoa dute, ...*

Adierazpenak: entitateen eta axiomen konbinazioak deskribapen konplexuak osatzeko: *funtzionarioa heldu bat da, gutxienez iraupen mugagabeko lan bat duena funtsa publikoak jasotzen dituen erakunde batean ...*

Entitateak

Banakoak (individuals): objektu zehatzak (Jon, Maria, ...)

Klaseak (classes): banakoak biltzen dituzten kategoriak (Pertsona, Emakumea, ...)

Propietateak (properties): entitateen arteko erlazioak (adinaDu, bikotea, ...)

Propietateak

- **Object properties**: objektua objektuarekin erlazionatzen dute (bikotea, ...)
- **Datatype properties**: datu-balioak objektuei esleitzen dizkiete (adinaDu, ...)
- **Annotation properties**: ereduaren parte ez diren metadatuak (rdfs:comment, rdfs:label, ...)

Arrazonamendua

Arrazonatzaileek (reasoners) automatikoki kalkula ditzakete axiomen **ondorioak**

A axioma-multzo batek a axioma **inplikatzen du** (entails) a egia bada A egia den bakoitzean

Axioma-multzo bat **konsistentea** izan daiteke (egoera posible bat dago non guztiak egiazkoak diren) edo **inkonsistentea**

OWL inkonsistentziak

Ontologia inkonsistentea: kontradikzio logikoak ditu

Klase ezin betegarria (Non satisfiable): ezin du instantziarik izan (owl:Nothing-en baliokidea)

Arrazonatzailea erabili inkonsistentziak detektatu eta azaltzeko

OWL 2 Sintaxia

Functional-Style Syntax: espezifikazio eta inplementaziorako

RDF/XML: OWL 2 tresna guztientzat derrigorrezkoa

Turtle: RDF serializazio irakurgarria

Manchester OWL Syntax: gizakientzat

OWL/XML: XML Schema-n oinarritua

Klaseak, Propietateak eta Banakoak

OWL 2-rekin oinarrizko modelaketa

[protege]

Klaseak eta instantziak

Banako bat klase batekoa dela deklaratzea:

```
:Mary rdf:type :Person .
:Mary rdf:type :Woman .
```

[OWL RDF Turtle syntax]

Banako bat hainbat klasetakoa izan daiteke aldi berean (RDF! Web Semantikoa! NoSQL!)

Klaseen hierarkia (SubClassOf)

"Emakume guztiak pertsonak dira" → azpiklase-erlazioa

```
:Woman rdfs:subClassOf :Person .
:Mother rdfs:subClassOf :Woman .
```

Klase baliokideak

Bi klaseek banako berberak dituzte:

```
:Person owl:equivalentClass :Human .
```

SubClassOf bi norabideetan deklaratzea bezalakoa da

Klaseen disjuntzioa (DisjointClasses)

Ezin da banakorik egon bi klaseetakoa dena:

```
:Woman owl:disjointWith :Man .
```

Maria Emakumea bada, arrazonatzaileak ondorioztatzen du Maria **ez** dela Gizona

[Arrazonatzailea aktibatu; inkonsistentea; explain]

Object Properties

Banakoak elkarren artean erlazionatzen dituzte (Positiboa edo negatiboa):

```
:Jon rdf:type owl:NamedIndividual ;
    :pareja :Mary .

[ rdf:type owl:NegativePropertyAssertion ;
  owl:sourceIndividual :Jon ;
  owl:assertionProperty :pareja ;
  owl:targetIndividual :Mary
] .
```

[Arrazonatzailea aktibatu]

Propietateen hierarkia

```
:pareja rdfs:subPropertyOf :relacion .
```

B A-ren bikotea bada → B-k A-rekin erlazioa du

(Alderantzizko erlazioa ez da nahitaez egia)

[Arrazonatzailea aktibatu]

Domain eta Range

Propietate batek zer entitate mota konektatzen dituen adierazteko:

```
:tieneHijo rdfs:domain :persona .
:tieneHijo rdfs:range :menor .
```

"Pedro tieneHijo Mikel (Adulto klasea)" bada → inferentzia?

Domain eta Range

Adulto disjointWith menor → inferentzia?

"Pedro tieneHijo Maria" bada → inferentzia?

⚠ Domain/Range **ez dira murrizketak**, **inferentzia-axiomak** dira

⚠ Domain/Range-rekin ohiko errorea

Adibidea: edad domain Pertsona

"Felix edad 9" eta "Felix rdf:type Katua" esaten badugu

→ Arrazonatzaileak ondorioztatzen du Felix **baita ere** Pertsona dela

Domain/Range-k ez dute balioztatzen: klase berriak **inferitzen** dituzte

(Irtenbidea?)

Banakoen berdintasuna eta ezberdintasuna

OWLek **ez du suposatzen** izen desberdinak = banako desberdinak (No Unique Name Assumption)

```
:James owl:sameAs :Jim .
:John owl:differentFrom :Bill .
```

Datatype Properties

Banakoak datu-balioekin (literalak) erlazionatzen dituzte:

```
:John :hasAge "51"^^xsd:integer .

[] a owl:NegativePropertyAssertion ;
  owl:sourceIndividual :Jack ;
  owl:assertionProperty :hasAge ;
  owl:targetValue "53"^^xsd:integer .

:hasAge rdfs:domain :Person .

:hasAge rdfs:range xsd:nonNegativeInteger .
```

XML Schema-ren datatypes: xsd:integer, xsd:string, xsd:boolean, …

Klaseen erlazio aurreratuak

Klase konplexuetarako eraikitzaile logikoak

Klaseen intersekzioa (AND)

Klaseak **bi** klaseetan dauden banakoak ditu:

```
EquivalentClasses(:Mother ObjectIntersectionOf(:Parent :Woman)) ] .
```

[OWL functional syntax]

→ Mother guztiak Woman dira eta Mother guztiak Parent dira

Klaseen bilketa (OR)

Klaseak **gutxienez klase bateko** banakoak ditu:

```
Class: Parent
  EquivalentTo:
    Father or Mother
```

[Manchester OWL syntax]

Klaseen osagarria (NOT)

Klasekoak **ez** diren banakoak:

```
# ChildlessPerson = Person AND NOT Parent

:ChildlessPerson owl:equivalentClass [
  owl:intersectionOf ( :Person [
    rdf:type owl:Class ;
    owl:complementOf :Parent
  ] )
] .
```

SubClassOf adierazpen konplexuekin

Beharrezko baldintzak baina ez nahikoak:

```
# Grandfather guztiak Man AND Parent dira
# (baina Man AND Parent guztiak ez dira Grandfather)
SubClassOf(
  :Grandfather
  ObjectIntersectionOf( :Man :Parent )
)
```

EquivalentClasses = **beharrezko eta nahiko** baldintza

SubClassOf = **beharrezko** baldintza soilik

Murrizketa existentziala (∃, SOME)

Gutxienez erlazionatutako balio bat klase batekoa da:

```
# Parent = gutxienez seme/alaba bat du Person dena
EquivalentClasses(
  :Parent
  ObjectSomeValuesFrom( :hasChild :Person )
)
```

Hizkuntza naturaleko adierazleak: "bat", "gutxienez bat"

Murrizketa unibertsala (∀, ONLY)

Erlazionatutako balio **guztiak** klase batekoak dira:

```
# HappyPerson = bere seme-alaba guztiak HappyPerson dira
EquivalentClasses(
  :HappyPerson
  ObjectAllValuesFrom( :hasChild :HappyPerson )
)
```

Adierazleak: "soilik", "esklusiboki", "beste ezer ez"

⚠ Murrizketa unibertsalaren tranpa

ObjectAllValuesFrom(:hasChild :HappyPerson)

Semerik **gabeko** pertsona batek murrizketa hau betetzen du!

Gainera gutxienez seme zoriontsu bat izatea exigitzeko:

```
EquivalentClasses(
  :HappyPerson
  ObjectIntersectionOf(
    ObjectAllValuesFrom( :hasChild :HappyPerson )
    ObjectSomeValuesFrom( :hasChild :HappyPerson )
  )
)
```

HasValue eta HasSelf

```
# John-en semeak = aita John dutenak
EquivalentClasses(
  :JohnsChildren
  ObjectHasValue( :hasParent :John )
)

# Nartzisista = bere burua maite du
EquivalentClasses(
  :NarcisticPerson
  ObjectHasSelf( :loves )
)
```

Kardinalitate-murrizketak

Erlazionatutako banako kopurua zehaztea:

```
# John-ek gehienez 4 seme-alaba ditu Parent direnak
ClassAssertion(
  ObjectMaxCardinality( 4 :hasChild :Parent )
  :John )

# John-ek gutxienez 2 seme-alaba ditu Parent direnak
ClassAssertion(
  ObjectMinCardinality( 2 :hasChild :Parent )
  :John )

# John-ek zehazki 3 seme-alaba ditu Parent direnak
ClassAssertion(
  ObjectExactCardinality( 3 :hasChild :Parent )
  :John )
```

Kardinalitate-murrizketak: Turtle

```
# Zehazki 5 seme-alaba
:John rdf:type [
  rdf:type owl:Restriction ;
  owl:onProperty :hasChild ;
  owl:cardinality "5"^^xsd:nonNegativeInteger
] .

# Gehienez 4 seme-alaba Parent direnak
:John rdf:type [
  rdf:type owl:Restriction ;
  owl:onProperty :hasChild ;
  owl:maxQualifiedCardinality
    "4"^^xsd:nonNegativeInteger ;
  owl:onClass :Parent
] .
```

Banakoen enumerazioa (OneOf)

Klase bat bere kide guztiak zerrendatuz definitzea:

```
# Klase itxia: Bill, John eta Mary soilik
EquivalentClasses(
  :MyBirthdayGuests
  ObjectOneOf( :Bill :John :Mary )
)

# Turtle:
:MyBirthdayGuests owl:equivalentClass [
  owl:oneOf ( :Bill :John :Mary )
] .
```

Jeff MyBirthdayGuests bada → Jeff = Bill, John edo Mary

Adierazpen konplexuak klaseetan

Adibideak/ExpresionCompleja.ttl

ONTOLOGY DESIGN PATTERNS (ODPs) PUBLIC CATALOG (Adibideak/odp_1.6)

Propietateen erabilera aurreratua

Ezaugarri eta eraikuntza aurreratuak

Alderantzizko propietateak

```
# hasParent hasChild-en alderantzizkoa da
InverseObjectProperties( :hasParent :hasChild )

# Turtle:
:hasParent owl:inverseOf :hasChild .

# Erabilera anonimoa adierazpenetan:
EquivalentClasses(
  :Orphan
  ObjectAllValuesFrom(
    ObjectInverseOf( :hasChild )
    :Dead
  )
)
```

Propietate simetrikoak eta asimetrikoak

```
# A hasSpouse B bada → B hasSpouse A
SymmetricObjectProperty( :hasSpouse )

# A hasChild B bada → INOIZ EZ B hasChild A
AsymmetricObjectProperty( :hasChild )

# Turtle:
:hasSpouse rdf:type owl:SymmetricProperty .
:hasChild rdf:type owl:AsymmetricProperty .
```

Asimetrikoa ≠ ez-simetrikoa (asimetrikoa indartsuagoa da)

Propietate erreflexiboak eta irreflexiboak

```
# Banako guztiak bere buruaren senideak dira
ReflexiveObjectProperty( :hasRelative )

# Inor ezin da bere buruaren aita izan
IrreflexiveObjectProperty( :parentOf )

# Turtle:
:hasRelative rdf:type owl:ReflexiveProperty .
:parentOf rdf:type owl:IrreflexiveProperty .
```

Propietate funtzionalak

```
# Pertsona bakoitzak gehienez senar bat du
FunctionalObjectProperty( :hasHusband )

# Senar bakoitza gehienez pertsona batena da
InverseFunctionalObjectProperty( :hasHusband )

# Turtle:
:hasHusband rdf:type owl:FunctionalProperty .
:hasHusband rdf:type owl:InverseFunctionalProperty .
```

Mary hasHusband James eta Mary hasHusband Jim bada → James = Jim

Propietate iragankorrak

```
# A hasAncestor B eta B hasAncestor C bada
# → A hasAncestor C
TransitiveObjectProperty( :hasAncestor )

# Turtle:
:hasAncestor rdf:type owl:TransitiveProperty .
```

hasParent hasAncestor-en azpipropietatea da: gurasoen gurasoen gurasoak automatikoki sartzen dira

(Adibideak/TransitivaFuncional)

Propietate disjuntuak

```
# Inor ez da pertsona beraren aita eta semea
DisjointObjectProperties( :hasParent :hasSpouse )
```

Ezin dira bi banako aldi berean bi propietateen bidez konektatuta egon

Propietate-kateak (Property Chains)

```
# hasParent o hasParent = hasGrandparent
SubObjectPropertyOf(
  ObjectPropertyChain( :hasParent :hasParent )
  :hasGrandparent
)

# hasFather o hasBrother = hasUncle
SubObjectPropertyOf(
  ObjectPropertyChain( :hasFather :hasBrother )
  :hasUncle
)
```

Gakoak (Keys)

```
# Gizarte Segurantzaren zenbakia Person bakoitza
# modu bakarrean identifikatzen du
HasKey( :Person () ( :hasSSN ) )

# Turtle:
:Person owl:hasKey ( :hasSSN ) .
```

Klasearen instantzia izendatu bakoitza propietate horien balioen bidez modu bakarrean identifikatzen da

Datatype-n erabilera aurreratua

Datatype-n murrizketak eta definizioak

Datatype-n definizioa

```
# Adina 0 eta 150 artean (XML Schema-ren fazetak)
DatatypeDefinition(
  :personAge
  DatatypeRestriction( xsd:integer
    xsd:minInclusive "0"^^xsd:integer
    xsd:maxInclusive "150"^^xsd:integer
  )
)

# majorAge = personAge AND NOT minorAge
DatatypeDefinition(
  :majorAge
  DataIntersectionOf(
    :personAge
    DataComplementOf( :minorAge )
  )
)
```

Datuen enumerazioa eta murrizketak

```
# toddlerAge = {1, 2}
DatatypeDefinition(
  :toddlerAge
  DataOneOf( "1"^^xsd:integer "2"^^xsd:integer )
)

# hasAge funtzionala da: pertsona bakoitzak adin bakarra du
FunctionalDataProperty( :hasAge )

# Teenager: adina 13 eta 19 artean
SubClassOf(
  :Teenager
  DataSomeValuesFrom( :hasAge
    DatatypeRestriction( xsd:integer
      xsd:minExclusive "12"^^xsd:integer
      xsd:maxInclusive "19"^^xsd:integer
    )
  )
)
```

Anotazioak eta ontologien kudeaketa

Metadatuak eta antolaketa

Anotazioak (Annotations)

Anotazioek **ez dute esanahi logikorik**: annotation property-en bidez kodetutako metadatuak dira (rdfs:label, rdfs:comment, ...)

Entitateak anotatu

Axiomak anotatu

Ontologiak anotatu

Ontologien kudeaketa

URI ontologia: Adibideak/ehu.ttl, URI entitateak?

Ontologiak fitxategien bidez inportatu: Adibideak/ehu-pdi.ttl, Adibideak/ehu-estudiante.ttl

Entitateak ontologien artean axiomen bidez lotu: owl:sameAs

Adibideak/ehu.ttl-n

Ontologien kudeaketa

Entitateen bateratzea URIen bidez: schema:Person

Ontologiak URIen bidez inportatu: LOV; Category tab: academy; edu-

Education Ontology

OWL 2 DL eta OWL 2 Full

OWL 2 interpretatzeko bi modu

OWL 2 DL

Direct Semantics:

- Sintaktikoki mugatutako azpimultzoa
- Erabakigarria: arrazonatzaileek erantzuna bermatzen dute
- Kalitatezko arrazonatzaile anitz eskuragarri

OWL 2 Full

RDF-Based Semantics:

- RDFSren luzapen naturala
- Erabakiezina: ez dago arrazonatzaile osorik
- Guztiz erreflexiboa (dena hirukote bat da)

OWL 2 Profiles

OWL 2 Profiles

OWL 2 osoa oso adierazgarria da baina konputazionalki garestia

Profilak propietate konputazional onak dituzten **azpimultzo sintaktikoak** dira

Profil bat ez da bestearen azpimultzoa

OWL 2 DL arrazonatzaile guztiak hiru profiletarako baliozkoak dira (baina motelago izan daitezke)

OWL 2 EL (Existential Language)

Optimizatua: deskribapen estruktural konplexuak dituzten ontologia handientzat: SNOMED-CT, Gene Ontology

Onartzen du: klase-adierazpen aberatsak, propietate-kateak, intersekzioa

Ez du onartzen: ezeztapena, disjuntzioa, kuantifikazio unibertsala, alderantzizkoak

Konplexutasuna: polinomikoa (PTIME)

OWL 2 QL (Query Language)

Optimizatua: datu-bolumen handien gaineko kontsultentzat

DL-Lite-n oinarritua: kontsulten itzulpena SQLra

Onartzen du: alderantzizkoak, propietate-hierarkiak, ER/UML diagramak

Ez du onartzen: kuantifikazio existentziala klase-adierazpenera, propietate-kateak, berdintasuna

Konplexutasuna: LogSpace datuetan (NLOGSPACE)

OWL 2 RL (Rule Language)

Optimizatua: arau bidezko arrazonamendu eskalagarria

Bateragarria: RDF(S), arauak (forward/backward chaining)

Onartzen du: intersekzioa, propietate-kateak, gehienezko kardinalitatea

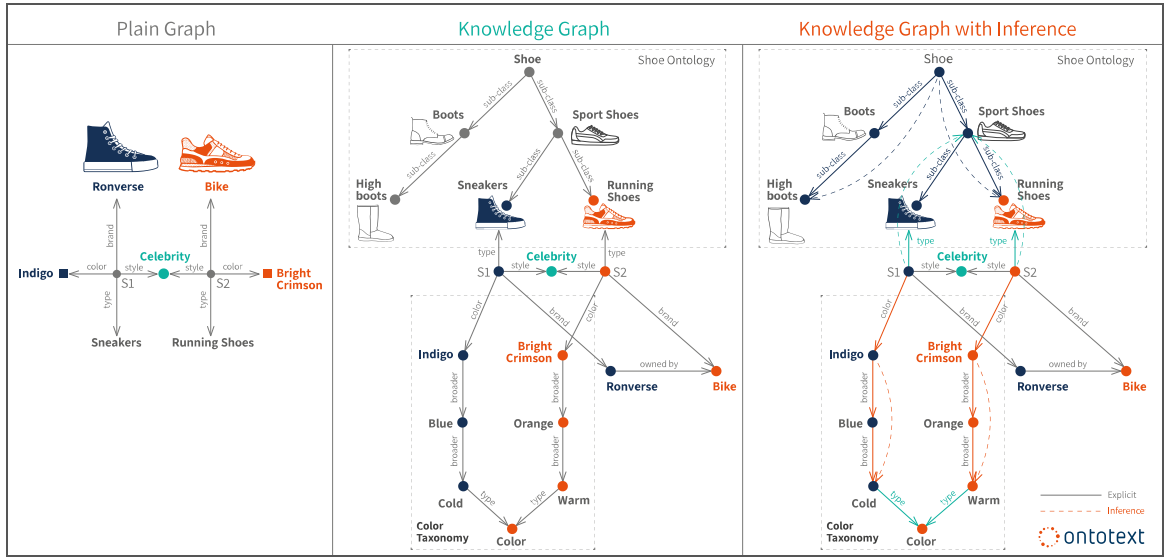
Ez du onartzen: banako ezezagunen existentzia behartzen duten existentzialak

Konplexutasuna: polinomikoa (PTIME)

OWL 2 profilen konparaketa

Profila	Erabilera-kasua	Inplementazioa
OWL 2 EL	Ontologia handien sailkapena	Arrazonatzaile espezializatuak (ELK, CEL)
OWL 2 QL	Kontsultak + ontologia arina	SPARQL → SQL berridazketa (Ontop)
OWL 2 RL	RDF datuen aberastea	Arau-motorrak (GraphDB)

Knowledge Graphs



OWL eta SPARQL

SPARQLek RDF grafoa kontsultatzen du, berez ez ditu inferitutako axiomak kontuan hartzen

GraphDB inference on: inferentziak materializatzen ditu (hirukote berriak) edo arrazonamendua aplikatzen du SPARQL emaitza hobeak lortzeko

Kontsulta federatuak interoperableagoak dira endpoint-ek ontologiak partekatzen badituzte

Ontologia publikoak

Ontologia-biltegiak: Linked Open Vocabularies, NCBO Bioportal, …

Schema

EU vocabularies: European Legislation Identifier, …

…